
FATTIBONI, ÁLVARO: 13201

PARCIAL I: CONTROL Y SISTEMAS

Contexto

Estás analizando la cantidad de turnos atendidos en un consultorio médico. El sistema registra diariamente el número de pacientes atendidos, generando una serie temporal discreta $x[k]$, donde cada muestra representa un día.

Objetivo

El equipo de gestión solicita elaborar un reporte resumido con un **valor representativo cada 10 días**, con el objetivo de facilitar el monitoreo operativo y la planificación de recursos a largo plazo, eliminando las fluctuaciones diarias irrelevantes para la tendencia general anual.

Criterio de evaluación

“No se evaluará únicamente el resultado numérico, sino la calidad del razonamiento técnico y la justificación de las decisiones de diseño.”

"Cuide la presentación, escala, colores, títulos y leyendas de sus gráficos para hacerlos lo más legible posible para la evaluación."

Tareas a desarrollar

Diseñar e implementar en MATLAB un script que permita reducir la serie temporal original. La salida debe ser una nueva serie reducida que entregue un valor representativo cada 10 días.

1. Análisis Previo de los Datos

Seleccione y aplique comandos de MATLAB para estudiar la señal original y extraer la información relevante.

```
clc
clear
x = load('datos_turnos.txt');

% Definimos la frecuencia de muestreo
Fs = 1; % 1 muestra/día. Frecuencia en [ciclos/día]

% --- Análisis en el dominio del tiempo ---
N = length(x);
t = 0:(N-1); % Vector de tiempo en días

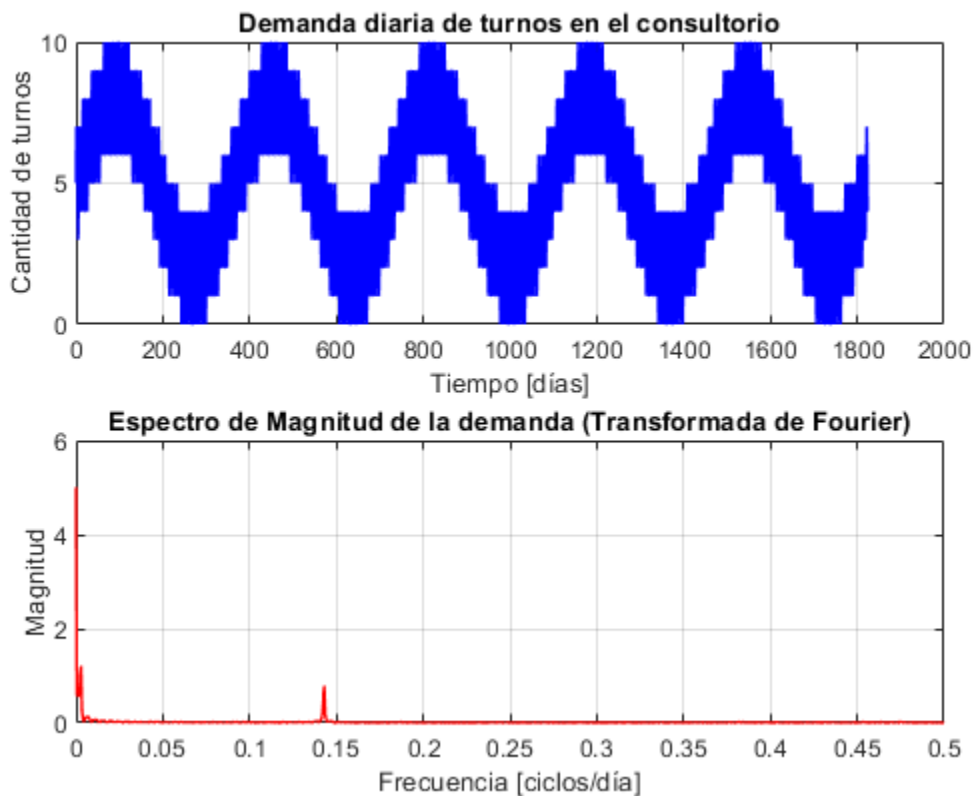
figure('Name', 'Análisis Previo: Tiempo y Frecuencia', 'Color', 'w');
subplot(2,1,1);
plot(t, x, 'b', 'LineWidth', 1);
title('Demanda diaria de turnos en el consultorio');
```

```
xlabel('Tiempo [días]');
ylabel('Cantidad de turnos');
grid on;

% --- Análisis en el dominio de la frecuencia ---
[f, dft_mag, dft_phase, dft, NFFT] = my_dft(x, Fs);

subplot(2,1,2);
plot(f, dft_mag, 'r', 'LineWidth', 1);
title('Espectro de Magnitud de la demanda (Transformada de Fourier)');
xlabel('Frecuencia [ciclos/día]');
ylabel('Magnitud');
xlim([0 Fs/2]); % Mostramos hasta la frecuencia de Nyquist (0.5 ciclos/día)
grid on;

%-----
```



1.1 Justifique brevemente la elección de dichas herramientas

% Respuesta

%Se utilizó un plot para ver la señal del archivo txt y luego se aplicó my_dft para ver qué frecuencias componen a la señal dato (con el ruido incluido).

1.2 Explique los resultados obtenidos del análisis

% Respuesta

%Se puede ver en el tiempo que existe una señal de baja frecuencia que tiene montada una señal de
%alta frecuencia, mientras que en el dominio de la frecuencia se puede ver la componente del ruido
%(alta frecuencia) bien alejada de la señal útil.

1.3 Mencione qué información le resulta relevante y útil para la solución.

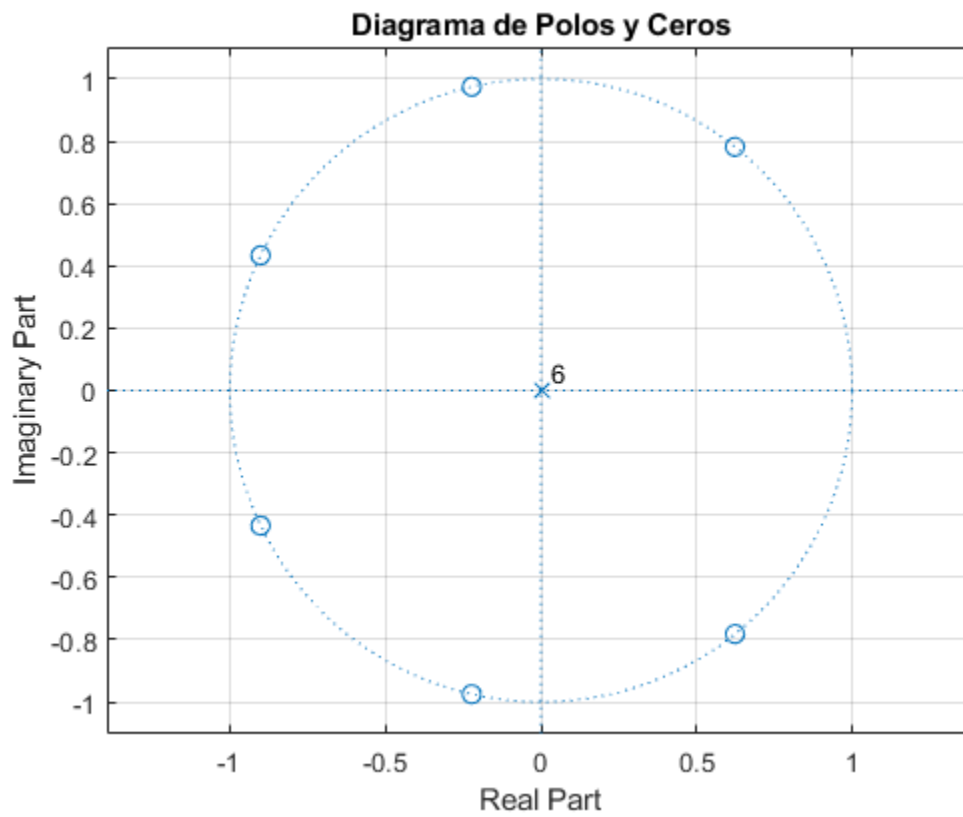
% Respuesta

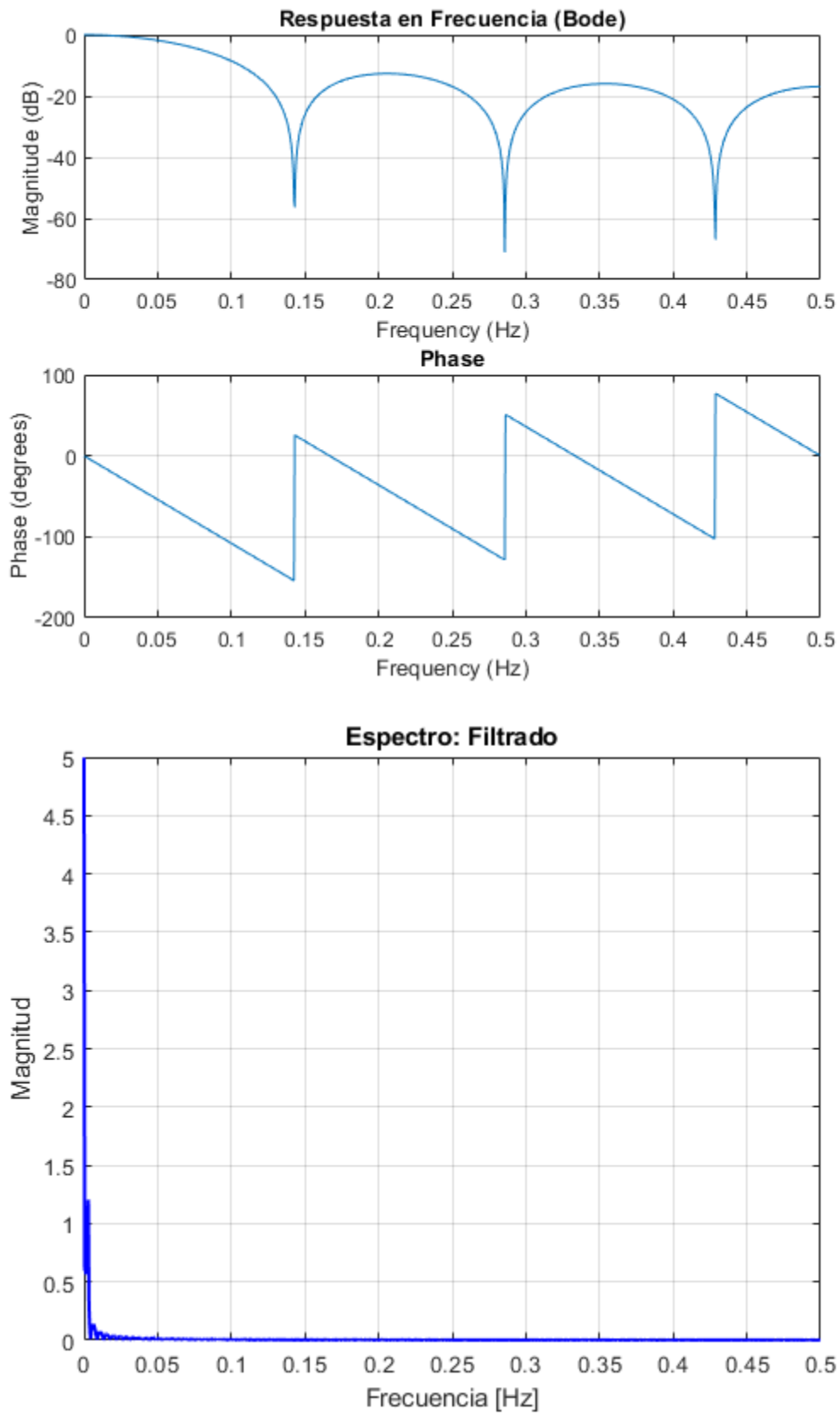
2. Pre-Proceso

Proceda a realizar el tratamiento de la señal

```
%-----  
% Coloque el código de su solución aquí  
%-----  
  
M = 10; % Factor de decimación (1 valor cada 10 días)  
N = 7; % Tamaño de la ventana del filtro Media Móvil (2 semanas exactas)  
  
% Diseño del filtro FIR pasa-bajos (Media Móvil)  
b = (1/N) * ones(1, N);  
a = 1;  
  
% Filtrado antialiasing de fase cero (garantiza alineación temporal perfecta)  
x_filtrada = filter(b, a, x);  
  
% Decimación: reducción de la tasa de muestreo  
x_reducida = downsample(x_filtrada, M);  
t_reducido = downsample(t, M); % t fue creado en la Sección 1  
  
% graficos de verificación  
  
figure;  
  
% Diagrama de Polos y Ceros  
zplane(b, a);  
title('Diagrama de Polos y Ceros');  
grid on;  
  
figure;  
% Diagrama de Bode (Magnitud y Fase)  
  
freqz(b, a, 1024, Fs);  
title('Respuesta en Frecuencia (Bode)');  
%-----  
  
[freq_filt, mag_filt] = my_dft(x_filtrada, Fs);  
  
figure('Name', 'Señal Senoidal Filtrada');  
plot(freq_filt, mag_filt, 'b', 'LineWidth', 1.5); hold on;
```

```
grid on
title('Espectro: Filtrado');
xlabel('Frecuencia [Hz]'); ylabel('Magnitud')
```





2.1 Comente brevemente sobre el método elegido para el pre-procesamiento.

```
% Se utiliza un filtro FIR pasa bajos de media movil haciendo coincidir el
primer cero del seno cardinal con la
% frecuencia del ruido (al estilo mata frecuencia) y luego se hace la
decimación solicitada.
```

2.2 ¿Por qué dicha técnica es necesaria y adecuada para este problema específico? (considerando el contenido frecuencial).

```
% Respuesta
%Es necesaria para evitar que las rápidas fluctuaciones diarias no afecten a
la señal de interés.
```

2.3 Mencione los criterios utilizados para definir los parámetros del sistema de procesamiento.

```
% Respuesta
%Mediante la observación del espectro de frecuencias de la señal original se
pudo identificar la
%frecuencia de la señal de ruido y con ella utilizar un filtro de orden 1/
f_ruido, aprovechando que
%1/f_ruido es un número entero. El valor de decimación ya viene dado.
```

3. Generar serie final y gráficos

A partir del procesamiento anterior, generar la serie final y graficuela en el tiempo y la frecuencia. Asegurarse de que la alineación temporal sea correcta respecto a la señal original.

```
%-----
% Coloque el código de su solución aquí
%-----
```

3. Generar serie final y gráficos

```
%-----
% Coloque el código de su solución aquí
%-----
```

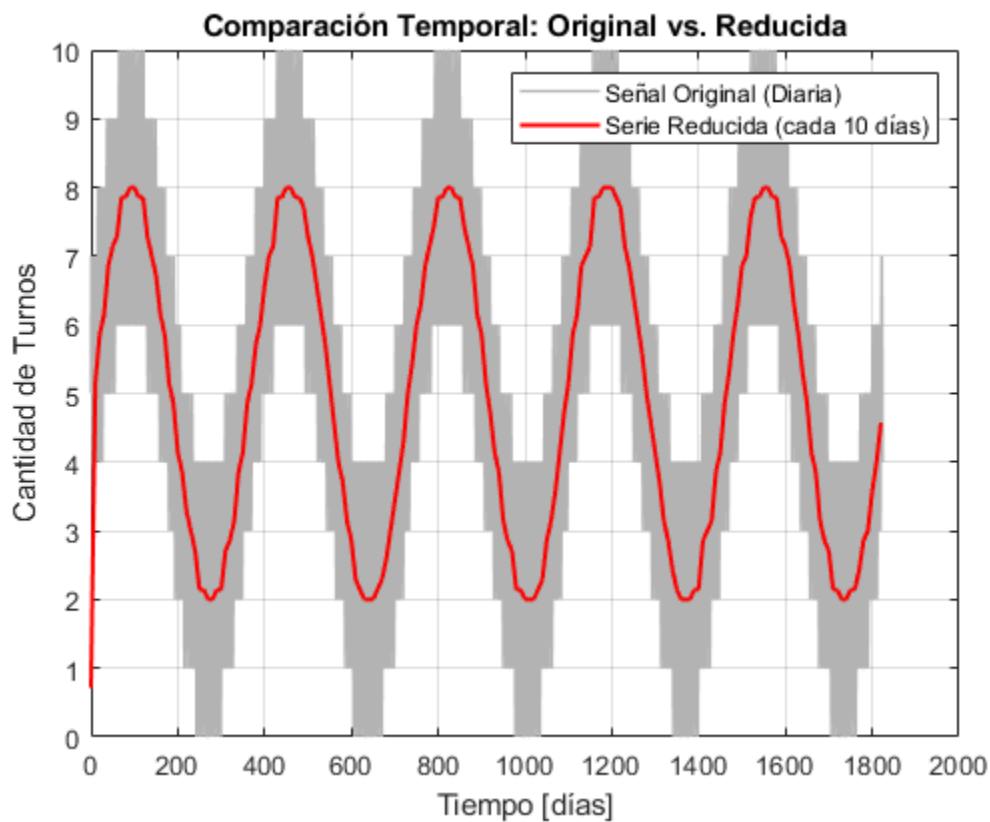
```
% 3.1. Cálculo de espectros para la validación en frecuencia
% Calculamos la TDF de la señal original y la señal final reducida
[f_orig, mag_orig] = my_dft(x, Fs);
[f_final, mag_final] = my_dft(x_reducida, Fs/M); % Fs_nueva = Fs/M

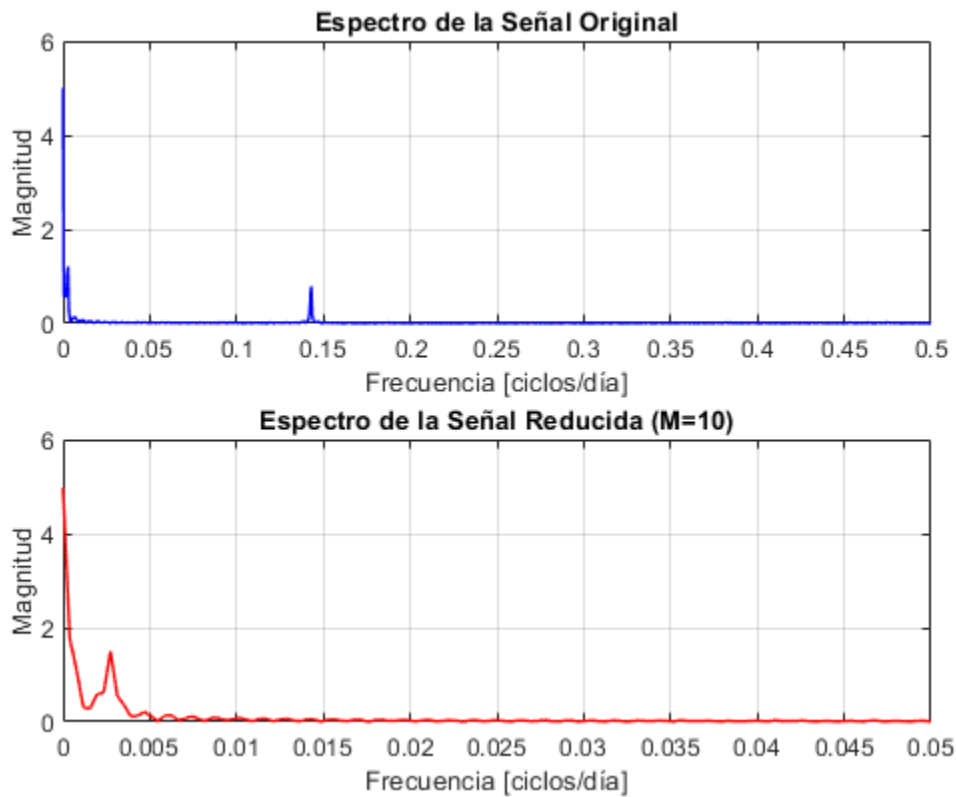
% 3.2. Gráfico en el dominio del Tiempo
figure('Name', 'Resultado Final: Dominio del Tiempo', 'Color', 'w');
plot(t, x, 'Color', [0.7 0.7 0.7], 'LineWidth', 1, 'DisplayName', 'Señal
Original (Diaria)');
hold on;
plot(t_reducido, x_reducida, 'r', 'LineWidth', 1.5, 'MarkerFaceColor', 'r',
'DisplayName', 'Serie Reducida (cada 10 días)');
title('Comparación Temporal: Original vs. Reducida');
xlabel('Tiempo [días]');
ylabel('Cantidad de Turnos');
legend('show');
```

```
grid on;
hold off;

% 3.3. Gráfico en el dominio de la Frecuencia
figure('Name', 'Resultado Final: Dominio de la Frecuencia', 'Color', 'w');
subplot(2,1,1);
plot(f_orig, mag_orig, 'b', 'LineWidth', 1);
title('Espectro de la Señal Original');
xlabel('Frecuencia [ciclos/día]');
ylabel('Magnitud');
xlim([0 Fs/2]);
grid on;

subplot(2,1,2);
plot(f_final, mag_final, 'r', 'LineWidth', 1);
title(['Espectro de la Señal Reducida (M=', num2str(M), ')']);
xlabel('Frecuencia [ciclos/día]');
ylabel('Magnitud');
xlim([0 (Fs/M)/2]); % Límite de Nyquist de la nueva señal
grid on;
%-----
%-----
```





4. Validación de la respuesta

Comparar su solución con al menos una alternativa que considere incorrecta o subóptima para validar por contraste la suya.

```
%-----  
% Coloque el código de su solución aquí  
%-----  
  
N2 = 5; % Tamaño de la ventana del filtro Media Móvil (2 semanas exactas)  
  
% Diseño del filtro FIR pasa-bajos (Media Móvil)  
b2 = (1/N2) * ones(1, N2);  
a2 = 1;  
  
% Filtrado antialiasing de fase cero (garantiza alineación temporal perfecta)  
x_filtrada2 = filter(b2, a2, x);  
  
% Decimación: reducción de la tasa de muestreo  
x_reducida2 = downsample(x_filtrada2, M);  
t_reducido2 = downsample(t, M); % t fue creado en la Sección 1  
  
%Graficos de verificación  
  
figure;
```

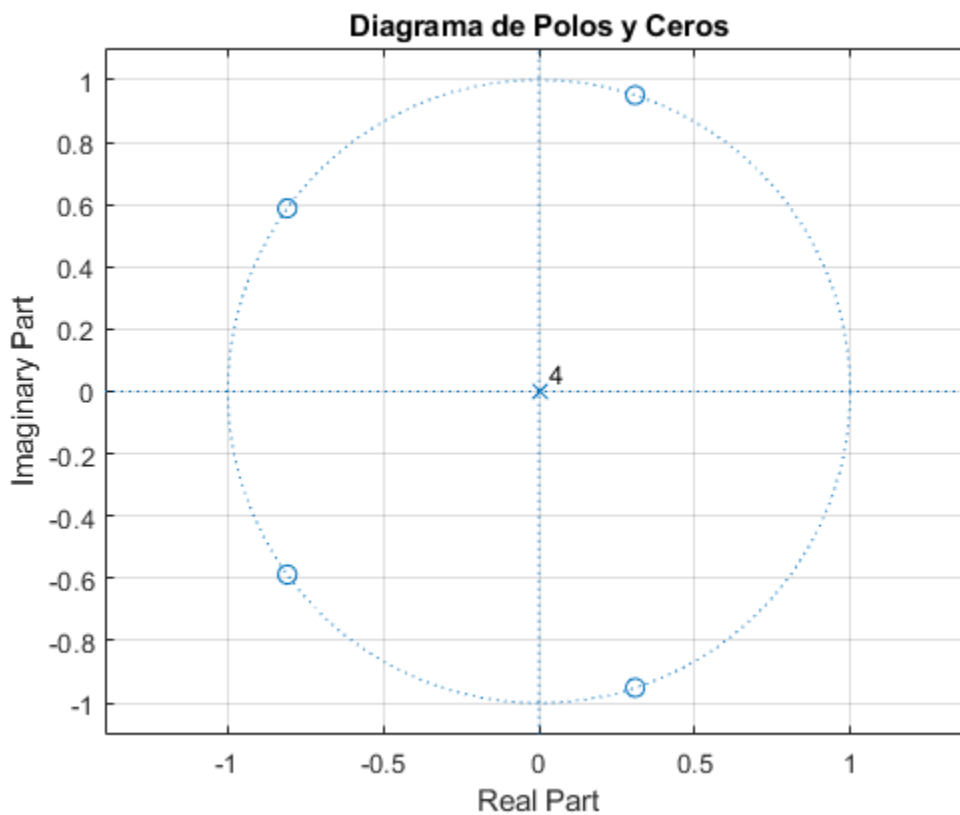
```
% Diagrama de Polos y Ceros
zplane(b2, a2);
title('Diagrama de Polos y Ceros');
grid on;

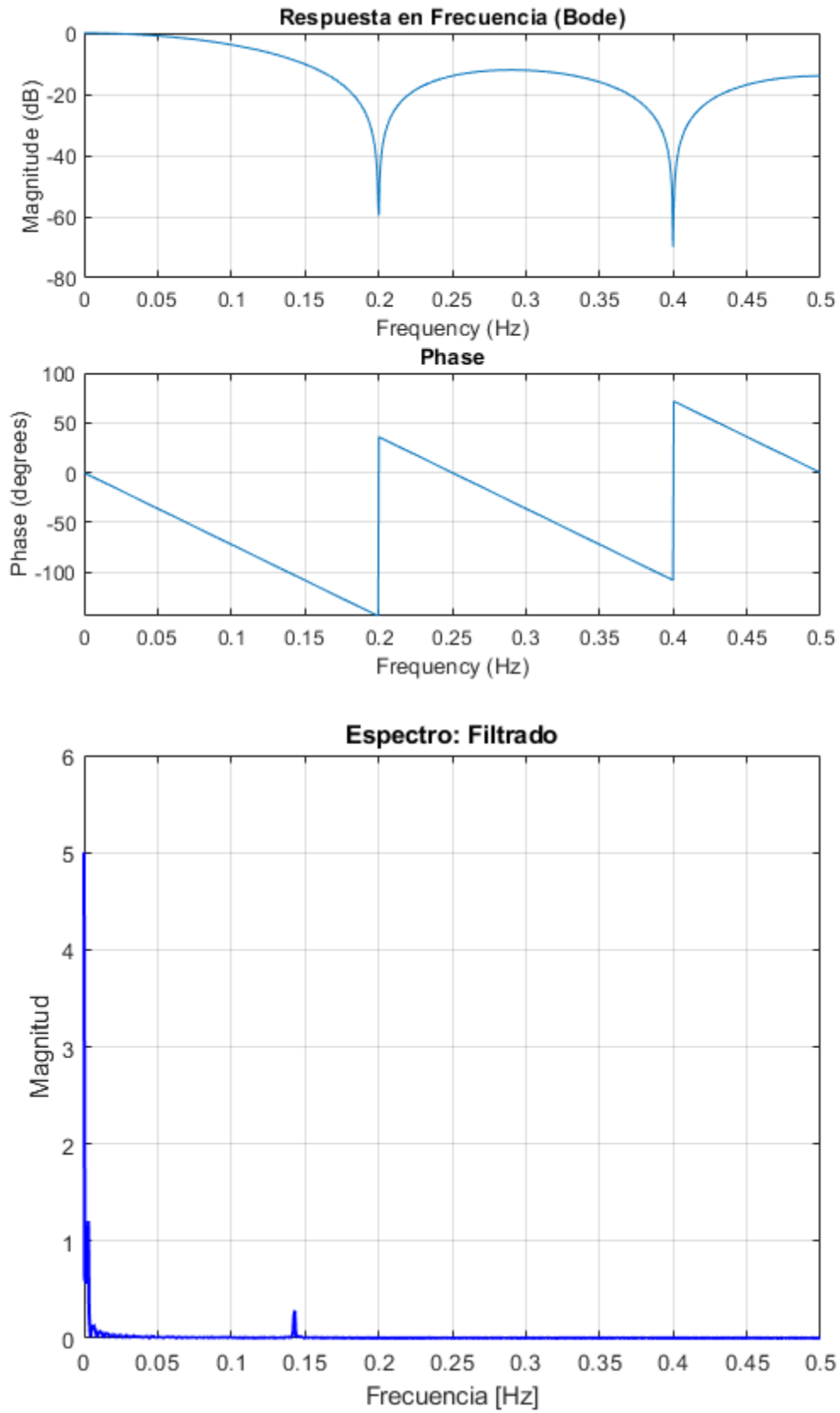
figure;
% Diagrama de Bode (Magnitud y Fase)

freqz(b2, a2, 1024, Fs);
title('Respuesta en Frecuencia (Bode)');
%-----

[freq_filt2, mag_filt2] = my_dft(x_filtrada2, Fs);

figure('Name', 'Señal Senoidal Filtrada');
plot(freq_filt2, mag_filt2, 'b', 'LineWidth', 1.5); hold on;
grid on
title('Espectro: Filtrado');
xlabel('Frecuencia [Hz]'); ylabel('Magnitud')
%-----
```





4.1 Qué efectos negativos produce la alternativa subóptima (en el dominio del tiempo y/o frecuencia).

```
% Respuesta
% No se alcanza a eliminar completamente el ruido.
```

4.2 Por qué su solución propuesta mitiga estos efectos.

```
% Respuesta

% Porque el orden del filtro es mayor, lo que mueve el cero del seno
cardinal.
```

5. Tasa de Cambio (Derivada) y Punto Fijo

Se nos comunica que conocer solo la cantidad total de pacientes ya no es suficiente para organizar el consultorio. Ahora necesitamos saber qué tan rápido sube o baja esa cantidad. Esta "velocidad de cambio" de la demanda se calcula directamente usando la derivada matemática.

Como ingeniero proponés usar **representación en punto fijo** para optimizar el sistema: buscamos aumentar la velocidad de cálculo y minimizar el uso de memoria.

1. Construya la velocidad de cambio de **su señal resultado del ejercicio anterior** en (turno/día).
2. Analice cómo se distribuyen los valores de su señal (usando un histograma o gráfico).
3. A partir de esa observación, determine un formato en punto fijo conveniente que ajuste para dicho rango
4. Indique de su respuesta: N tamaño de la palabra binaria (8,16 o 32 bits) y los m bits para la parte entera, n bits para la parte fraccionaria usando la notación Qm.n (con 1 bit de signo), rango, resolución y ruido cuantización

Completar con los resultados obtenidos:

- **Palabra binaria:** 8 bits
- **Representación:** Q0.7
- **Rango:** [-1, 0.9921875]
- **Resolución:** 0.015625
- **Ruido de cuantización:** +-0.0078125

5. Tasa de Cambio (Derivada) y Punto Fijo

```
%-----
% Cálculo de la velocidad
%-----
% La distancia temporal entre muestras de la serie reducida es dt = 10 días.
dt = 10;
velocidad = diff(x_reducida) / dt; % Unidades: [turnos / día]

% Ajustamos el vector de tiempo (al derivar, el vector se achica en 1
muestra)
t_velocidad = t_reducido(2:end);
```

```
%-----
% Análisis de distribución (Gráficos)
%-----
figure('Name', 'Sección 5: Velocidad de Cambio', 'Color', 'w');

subplot(2,1,1);
plot(t_velocidad, velocidad, '-o', 'Color', '#D95319', 'LineWidth', 1.5,
'MarkerFaceColor', '#D95319');
title('Velocidad de cambio de la demanda de turnos');
xlabel('Tiempo [días]');
ylabel('Velocidad [turnos/día]');
grid on;

subplot(2,1,2);
histogram(velocidad, 15, 'FaceColor', '#EDB120');
title('Histograma de distribución de la Velocidad');
xlabel('Velocidad [turnos/día]');
ylabel('Frecuencia absoluta');
grid on;

%-----
% Verificación de Punto Fijo
%-----
fprintf('\n--- ANÁLISIS DE PUNTO FIJO ---\n');
senal = velocidad; % Asignamos la derivada a tu variable de análisis

% Definición del formato (Buscamos minimizar memoria: N=8 bits -> 1 signo,
m=0, n=7)
m = 0; % Parte Entera
n = 7; % Parte Fraccionaria

% Datos físicos de la señal
sig_max = max(senal);
sig_min = min(senal);

% Cálculo del paso mínimo real de la señal (tu lógica)
diferencias = abs(diff(senal));
diferencias_reales = diferencias(diferencias > 0);
salto_minimo = min(diferencias_reales);

% Datos del punto fijo elegido
limite_inf = -2^m; % Limite inferior
limite_sup = 2^m - 2^-n; % Limite Superior
res = 2^-n; % Resolución (LSB o Quantum)

% Impresión de reporte
fprintf('Formato elegido: Q%d.%d (Palabra N = %d bits)\n', m, n, m+n+1);
fprintf('Señal real -> Max: %.4f, Min: %.4f, Salto Mínimo: %.4f\n', sig_max,
sig_min, salto_minimo);
fprintf('Q%d.%d -> Rango: [%.4f, %.4f], Resolución: %.4f\n\n', m, n,
limite_inf, limite_sup, res);

% Verificación automática
disp('Resultados de validación:');
```

```
if salto_minimo - res > 0
    disp(" > Resolución: SUFICIENTE (El bit menos significativo es menor al
menor cambio físico)");
else
    disp(" > Resolución: NO suficiente");
end

if limite_sup >= sig_max
    disp(" > Límite superior: SUFICIENTE (No hay overflow positivo)");
else
    disp(" > Límite superior: NO suficiente (Peligro de Overflow)");
end

if limite_inf <= sig_min
    disp(" > Límite inferior: SUFICIENTE (No hay overflow negativo)");
else
    disp(" > Límite inferior: NO suficiente (Peligro de Overflow)");
end

%-----

function [f, dft_mag, dft_phase, dft, NFFT] = my_dft(data, Fs)
% my_dft: devuelve la transformada discreta de Fourier de una serie de
tiempo.
%
% Si la unidad de data es U, la unidad de la TDF entregada es U/Hz.
%
% INPUT
%   data: NxD serie de tiempo.
%   Fs: 1x1 frecuencia de muestreo de data.
%
% OUTPUT
%   f      : Nx1 vector frecuencia, en Hz.
%   dft_mag : NxD magnitud de la transformada discreta de Fourier.
%   dft_phase : NxD fase de la transformada discreta de Fourier.
%   dft      : NxD transformada de Fourier discreta, valores complejos.
%   NFFT      : 1x1 cantidad de elementos usados en la FFT.
%
% Version: 003
% Date:      2019/05/08
% Author:    Rodrigo Gonzalez <rodraz@frm.utn.edu.ar>

[n, m] = size(data);

if (m > n)                                % Si data es un vector fila...

    data = data';                          % Trasponer data
    n = m;
end

L = n;                                    % Tamaño de la serie de tiempo

NFFT = 2^nextpow2(L);                     % Potencia de 2 siguiente al tamaño del
```

```

vector
f = Fs/2*linspace(0,1,NFFT/2+1);    % Vector frecuencia

dft = fft(data, NFFT) / L;           % Transformada rapida de Fourier

dft = dft(1:NFFT/2+1, :);            % Se toman solo las frecuencias positivas

dft_mag = abs(dft);                   % Magnitud
dft_phase = angle(dft);               % Fase

% dft_r = real(dft); % Parte real de la transformada discreta de Fourier
% dft_i = imag(dft); % Parte imaginaria de la transformada discreta de
Fourier

end

```

--- ANÁLISIS DE PUNTO FIJO ---

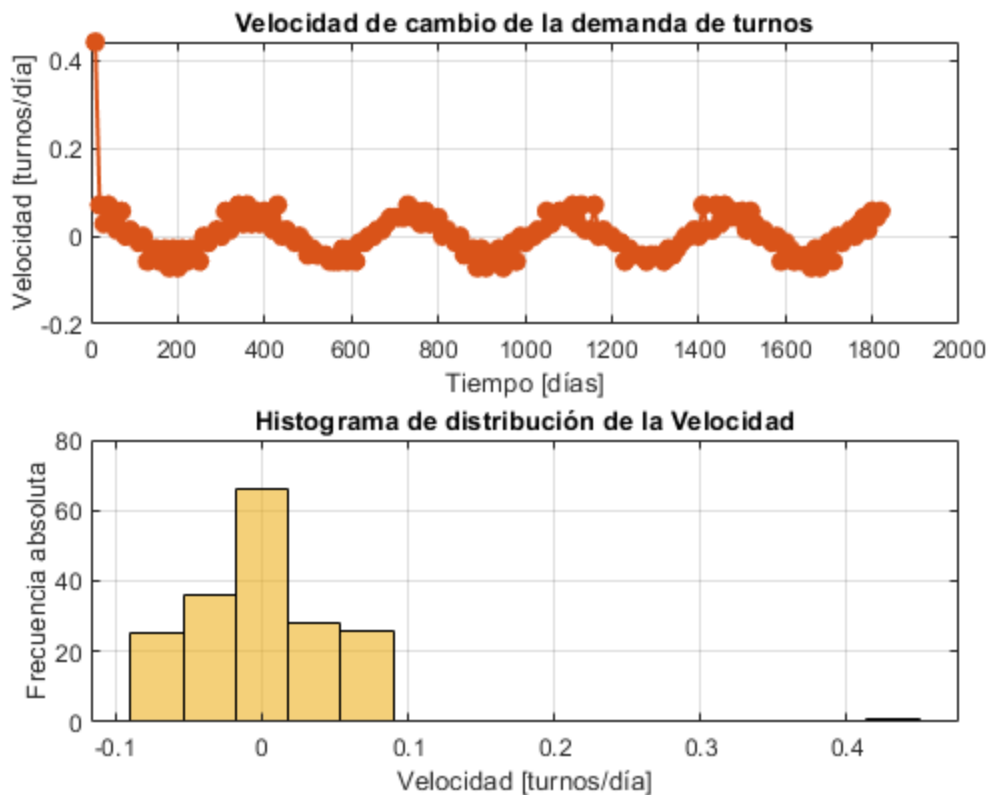
Formato elegido: Q0.7 (Palabra N = 8 bits)

Señal real -> Max: 0.4429, Min: -0.0714, Salto Mínimo: 0.0000

Q0.7 -> Rango: [-1.0000, 0.9922], Resolución: 0.0078

Resultados de validación:

- > Resolución: NO suficiente
- > Límite superior: SUFICIENTE (No hay overflow positivo)
- > Límite inferior: SUFICIENTE (No hay overflow negativo)



Published with MATLAB® R2023b